

TikHub 小白指南

一个 API Key 买通 17 个社交平台 · 1003 个端点 / 1007 个 MCP 工具 · 本指南 = 「社交数据采集三部曲」第三块拼图：Agent Reach (本地免费) → AgentKey (open-core 订阅) → TikHub (纯商业按量计费) | 来源：冰河推文 (拾遗队列) | 2026-10-02

Q1 TikHub 是什么？一句话说清

TikHub 是一个「社交数据超市」：你注册一个账号拿一个 API Key，就能调 1003 个现成的接口，实时读取 TikTok、抖音、Instagram、YouTube、Twitter/X、小红书、B站、微博等 17 个平台的公开数据。

它自己**不生产数据、不卖爬虫软件**——它把“去各平台抓公开数据”这件脏活累活统一成 HTTP 接口卖给你，按请求次数计费（\$0.001–0.01/次）。官方口径：3 万+ 开发者、每日 3 亿+ 请求、10 亿+ 条预采集数据集记录。它同时提供 REST API（给程序）和 MCP 服务器（给 AI Agent），后者是今天推文推荐的重点。



Q2 没有 TikHub 之前，要 17 个平台的数据得怎么做？

这就是知识库已学过的 **[[M×N集成问题]]**：你（1 个需求方）对 N 个平台，每家都要单独写爬虫 / 逆向接口 / 养 Cookie / 抗风控 / 改解析器——而且平台改一次版式，你的爬虫就死一次。三大成本：

成本	自己干	交给 TikHub
开发	17 套爬虫 + 17 套 JSON 解析	1 套统一 REST 接口，响应结构一致
运维	平台一改版就坏，自己 24h 救火	渠道腐化由厂商养 （它的生意就是保持接口活着）
风控	自己 IP 被封、账号被ban	代理池、指纹、验证码都是厂商的事

代价也直白：请求必经**第三方**（隐私/延迟/单点风险）+ 中间商抽成。这正是已学概念 **[[统一数据网关]]**（Master Key 模式）的商业化实现——vault 里那条概念的正文当时就已点名 tikhub，今天算是见到了正主。

Q3 「API 即端点」是什么设计？1003 个端点怎么组织？

所谓 "API 即端点": 每个采集能力就是一个独立的 REST 端点, 而不是一个万能接口带一堆参数。命名直接可读:

```
# 端点长这样 (示例命名模式):
fetch_one_video      # TikTok 单视频数据
fetch_user_info      # Instagram 用户信息
fetch_search_timeline # Twitter 搜索时间线
```

组织方式 = 按平台分域 + 按功能分组 (视频/用户/搜索/评论/广告/电商/创作者工具...)。实测 GET `https://mcp.tikhub.io/platforms` (免认证) 拿到各平台端点数:

平台	抖 音	TikTok	Instagram	微 博	小红 书	B 站	快 手	知 乎	YouTube	Twitter	Reddit	...
端点 数	319	162	93	56	45	42	38	34	34	25	28	共 1003

意外发现: 抖音端点最多 (319 个, 含星图/巨量/DouPlus 等营销工具) —— 中文平台数据反而是 TikHub 最厚的家底。

Q4 AI Agent 怎么用上它? ——MCP 三行接入

TikHub 把每个平台的端点包装成该平台专属的 MCP 服务器, 托管在 `mcp.tikhub.io/{platform}/mcp`, 共 1007 个工具。三种传输方式 (对应已学的 [[stdio与流式HTTP传输]]) :

传输	端点	适用客户端	Key 怎么传
Streamable HTTP	<code>/tiktok/mcp</code>	Cursor、Cline、curl、自定义	HTTP 头 <code>Authorization: Bearer <KEY></code>
SSE	<code>/tiktok/sse</code>	旧版流式客户端	同上 (也可自托管 <code>tikhub-mcp --platform tiktok</code>)
Stdio 桥接	本地进程	Claude Desktop / Claude Code / VS Code	<code>npx mcp-remote</code> 把 HTTP 桥成子进程, Key 走 <code>--header</code>

```
// Claude Desktop 配置 (claude_desktop_config.json) — 这就是"三行接入":
"tikhub-tiktok": {
  "command": "npx",
  "args": ["mcp-remote", "https://mcp.tikhub.io/tiktok/mcp",
    "--header", "Authorization: Bearer YOUR_API_KEY"]
}
```

LangChain/LangGraph 侧用 `langchain-mcp-adapters` 的 `MultiServerMCPClient` 挂多个平台服务器。接入协议层就是标准 MCP: initialize → tools/list → tools/call (底层 [[JSON-RPC与stdio传输]] 的远程版)。

Q5 1000+ 工具，不会把模型上下文撑爆吗？（本指南最有含金量的一问）

会爆。这正是新概念 **[[工具目录膨胀]]** (Tool Catalog Bloat)：MCP 客户端要把每个工具的名称+描述+参数 schema 全塞进系统提示词。TikHub 的解法只有一个，但实测有效：

按平台拆成 17 个独立 MCP 服务器，官方建议"只添加你用到的平台"。这样 Agent 的可见工具数 = 单平台数量而非全局 1007。

✗ **全量挂载**：1007 工具 $\approx$ 15万+ token 的目录 $\rightarrow$ 还没干活上下文先满

1007 个工具目录 $\approx$ 整个上下文窗口被说明书占满

✓ **按域拆分**：只挂 1 个平台 (实测 TikTok)

162 工具 $\approx$ 24,846 tokens (实测 99KB 目录 JSON $\div$ 4) $\approx$ 128k 窗口的 20%

对照已有概念：

[[分层按需加载]] (小入口常驻·大主体待命) · [[KV缓存与上下文]] (上下文是按 token 计费的稀缺资源) · 与 Agent-Reach 「默认只激活

✦ **边界要诚实**：TikHub 没有工具搜索/分页/按需加载机制——单平台内部 162~319 个工具仍是一次性 **全量 tools/list** (实测抖音平台会更大)。挂 2~3 个平台后同样会重新膨胀。它是"粗粒度拆分"，不是精细的上下文管理。

Q6 免费和付费的分界线在哪？——实验实测：控制面开放、数据面计费

定价很简单：注册送 **~50 次免费请求 + \$0.05**（无需信用卡），之后按量付费 \$0.001–0.01/次，最低充 \$5，日用量阶梯折扣最高 -50%，失败请求（非200）不计费。但真正有意思的是**闸门装在哪**——本指南 11 项真实探测的发现：



✓ **实测细节：**① 完全不带 Authorization 头 → HTTP 401 + `www-authenticate: Bearer realm="TikHub MCP"`（静态 Bearer，**没有** resource_metadata URL——对照 AgentKey 走的 [[OAuth资源发现]] RFC 9728 路线）；② 假 Key `tk_fake_key_123` → initialize、tools/list 全部 HTTP 200；③ tools/call → HTTP 200 的 MCP 成功信封里，`content[0].text` 装着 `{"error": "unauthorized", "status": 401}`。

两个由此提炼的知识点：[[工具目录膨胀]] 的免费橱窗策略（目录公开是获客手段）+ [[控制面与数据面分离]]（认证/计费闸门的位置是一种架构决策，MCP 把业务错误包进成功信封，让 LLM 读得懂失败原因并自我纠正）。

Q7 同赛道三兄弟怎么选？Agent Reach vs AgentKey vs TikHub

这是今天流水线连学三个同类工具的最大收获——**同一个需求（让 Agent 拿到互联网数据），三种商业/技术形态，复杂性的放置位置不同：**

	Agent Reach	AgentKey	TikHub
形态	本地安装器+能力层 (MIT 开源, 88k★)	open-core: 开源技能包 + 闭源云 MCP	纯商业 SaaS (仅 SDK 开源)
数据从哪来	直连上游开源 CLI (yt-dlp/gh/Jina...)	经厂商云 (28+ 平台)	经厂商云 (17 平台, 中文最深)
计费	免费 (自己养)	订阅 + credit	按请求计费 \$0.001-0.01/次
认证	本地 Cookie/Token 各渠道自管	OAuth 资源发现 (RFC 9728, 401引导)	静态 Bearer (401 直给提示)
控制面	—	initialize 就验权 (实测 401)	initialize/tools/list 开放 (实测假 Key 过)
上下文策略	默认激活 6 渠道 其余待命	find_tools 动态发现 (防工具名幻觉)	按平台拆 17 服务器 只挂你用的
适合谁	个人开发者 愿折腾、要免费	要省事、要稳定的 Agent 产品	规模化取数 数据集/训练/监控

一句话记住：Agent Reach 把复杂性留给硬盘，AgentKey 把复杂性卖进订阅，TikHub 把复杂性卖进按次账单。三者都在解 [[M×N集成问题]]，只是"谁替你养渠道"的收费姿势不同。

Q8 数据从哪来？合规吗？灰色地带在哪？

TikHub 口径：**只采集公开数据**（公开视频、公开主页、公开评论、弹幕），声明 GDPR/CCPA 合规、"不拥有/转售/再分发底层数据本身，只提供工具"。

但工科生要保持清醒：**"平台公开可见" ≠ "平台允许程序化批量采集"**。各平台服务条款普遍禁止未经授权的自动化抓取，TikHub 本质上是在替你做这件事并承担技术风险。商用前注意：① 目标平台的 ToS 风险自担；② 不要采个人隐私数据；③ 欧盟注意 GDPR 对"公开个人数据"仍有约束。把它当**研究与原型阶段的加速器**最稳妥。

Q9 除了 MCP 还有什么玩法？

- **REST API 直调**：不要 Agent 也能用，纯 HTTP + Bearer，官方 Python/Java SDK 开源在 GitHub (TikHub org)。
- **预采集数据集**：10 亿+ 条结构化记录 (CSV/JSON/JSONL/Parquet)，量大的话每千条低至 \$0.40——做 AI 训练/微调比逐条实时抓便宜得多。
- **Marketplace (2026-08 上线)**：第三方可以在上面卖自己的 API/数据集/Agent，佣金 10% (行业约一半) ——“数据能力商店”的野心。
- **Chrome 扩展**：自带 Key 的浏览器侧边栏，直接分析当前刷着的 TikTok/抖音/IG/X/B站内容。
- **自托管 MCP 分发**：`tikhub-mcp` 可本地跑 (.exe 分发)，把云 MCP 桥进只支持 stdio 的宿主。

动手实验：11 步真实探测 (无需注册，全部可复现)

实验设计思路：**不注册账号也能把协议摸透**——用免认证元端点 + 假 Key 行为分析，把 TikHub 的认证模型、工具目录、上下文体积全部实测出来。完整记录见 `tikhub/exercise/experiment-log.md`。

步骤 1：健康检查 (免认证)

操作：

```
curl https://mcp.tikhub.io/health
```

目的：确认服务活着、拿全局元信息。✅ **预期结果**：

```
{"status": "healthy", "version": "2.0.0", "platforms": 17, "total_endpoints": 1003}
```

步骤 2：拉全平台清单 (免认证)

操作：

```
curl https://mcp.tikhub.io/platforms
```

✅ **预期结果**：17 个平台的 slug、名称、描述、`endpoint_count` (抖音 319 / TikTok 162 / IG 93...) 和各自的 mcp_url / sse_url。实现：不花一分钱看清全平台货架。

步骤 3: 无 Key 调 MCP 端点 → 看网关 401 长什么样

操作:

```
curl -i -X POST https://mcp.tikhub.io/tiktok/mcp \
  -H "Content-Type: application/json" \
  -H "Accept: application/json, text/event-stream" \
  -d '{"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocolVersion":"2025-03-26","capabilities":{},"clientInfo":{"name":"probe","version":"1.0"}}}'
```

✅ 预期结果: HTTP 401 + 头 `www-authenticate: Bearer realm="TikHub MCP"` + JSON-RPC 错误体 `code:-32001`。注意: 没有 `resource_metadata` URL → 静态 Bearer, 不走 OAuth 发现 (对比 AgentKey)。

步骤 4: 假 Key initialize → 控制面是否放行?

操作: 同上请求, 加 `-H "Authorization: Bearer tk_fake_key_123"`。

✅ 预期结果: HTTP 200! 返回 SSE 事件流: `serverInfo: {"name":"TikHub TikTok MCP","version":"1.20.0"}` + 平台说明。响应头里拿 `Mcp-Session-Id` (步骤 5 要用)。

步骤 5: 假 Key tools/list → 工具目录免费可见

操作:

```
curl -X POST https://mcp.tikhub.io/tiktok/mcp \
  -H "Authorization: Bearer tk_fake_key_123" \
  -H "Mcp-Session-Id: <步骤4拿到的会话ID>" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json, text/event-stream" \
  -d '{"jsonrpc":"2.0","id":2,"method":"tools/list","params":{}}'
```

✅ 预期结果: 完整工具目录 (name + 双语 description + inputSchema + outputSchema)。

步骤 6: 量化工具目录膨胀

操作: 步骤 5 响应落盘后用 Python 统计:

```
import re, json
raw = open("toolslist_raw.txt", encoding="utf-8").read()
d = json.loads(re.search(r"data: (.*)", raw).group(1))
tools = d["result"]["tools"]
print(len(tools), len(json.dumps(tools, ensure_ascii=False)) // 4, "tokens")
```

✅ 预期结果: `162 24846 tokens` —— 单平台目录就吃掉 ~2.5 万 token (128k 窗口的 20%)，亲眼看见 [[工具目录膨胀]]。162 与 /platforms 报的 `endpoint_count` 精确一致。

步骤 7: 假 Key tools/call → 数据面闸门现身

操作: 同头调 `tiktok_web_fetch_post_detail` (给个真实视频 itemId)。

✓ 预期结果: HTTP 200 的 MCP 成功信封, 但 `content[0].text` 里是

`{"error": "unauthorized", "status": 401, "message": "TikHub API key was rejected..."}` —— 协议成功, 业务失败, 错误以工具结果回流, LLM 能读懂原因并自纠 ([[Agent 循环]] 的燃料)。

步骤 8-11 (补充探测): 边界行为

① 无 Key GET `/tiktok/sse` → 401; ② GET `/nonexistent/mcp` → 401 (认证先于路由, 不泄露平台存在性); ③ 带/不带 `Mcp-Session-Id` 对比; ④ 响应永远是 `event: message + data:` SSE 封装。全部真实跑通, 记录见 `experiment-log.md`。

✓ 总结: 控制面 (`initialize/tools/list`) 对任何人开放——工具目录是免费橱窗; 真 Key 校验发生在数据面 (`tools/call` 上游)。加上免认证元端点, 零成本完成一次完整的安全架构田野调查。

这个项目给我什么 (能力清单)

你能用它干的 5 件事 (按上手难度排序):

1. **零代码** 给 Claude/Cursor 装社交数据感官: 注册拿免费 Key → 三行 MCP 配置 → "帮我看看这条 TikTok 的评论风向" 直接可问 (~50 次免费请求够体验)。
2. **脚本级** Python 小监控: SDK + cron, 定时抓某账号/话题的新内容做简报——不用维护任何爬虫。
3. **架构参考** 抄它的 MCP 托管模式: 按域拆服务器 + 元端点 (`/health /platforms` 免认证) + 静态 Bearer——自己想发布 MCP 服务时的现成模板。
4. **数据级** 买训练数据集: 做垂直领域微调时, 按千条计价的预采集数据集比实时抓便宜一个量级。
5. **方法论** 学会"控制面/数据面"分析框架: 拿到任何付费 API/MCP 服务, 先用假 Key 探测闸门位置——免费边界在哪、错误以什么形态回流, 5 分钟摸清一家服务的商业与安全架构。

坑与结论

- ⚠️ 1. **目录不等于可用**：部分接口仅限付费，免费额度会直接被跳过扣付费余额（官方定价页明示）。
- ⚠️ 2. **"1007 工具"三处数字对不上很正常**：营销页 990+/1007、/health 1003 endpoints、分平台之和——统计口径（端点 vs 工具 vs 平台服务器）不同，引用时注明来源。
- ⚠️ 3. **响应永远 SSE 封装**：即使 Accept 带 application/json，Streamable HTTP 也回 `event: message\ndata:` 格式，解析时记得剥壳。
- ⚠️ 4. **会话有状态**：initialize 后的 `Mcp-Session-Id` 必须带在后续每个请求上，无 Key 阶段没有这个头。
- 💡 5. **三部曲的终极启示**：数据获取这件事的复杂性不会消失，只会转移——要么你自己养（Agent Reach）、要么付月费让厂商养（AgentKey）、要么按次付账单让厂商养（TikHub）。选型本质是**用哪种货币支付复杂性：时间、订阅、还是单价。**

TikHub 小白指南 · 拾遗队列 task 6fa3e8fb · 2026-10-02 · 实验全部真实运行（curl + python，零注册协议级探测）

关联：F:\reminder\agent-reach（本地能力层）· F:\reminder\agentkey（open-core 网关）· vault：[[工具目录膨胀]] [[控制面与数据面]] [[统一数据网关]] [[MCP协议]]